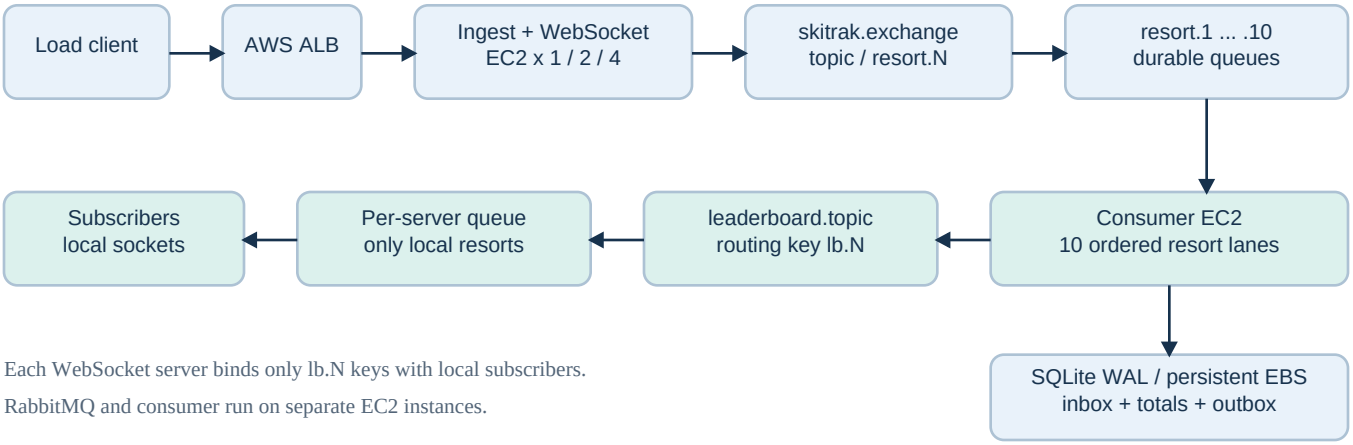
SkiTrak | Assignment 2

Architecture and delivery logic | Java 11 / Tomcat 9 | 10 resorts / Top-20



Each WebSocket server binds only lb.N keys with local subscribers.

RabbitMQ and consumer run on separate EC2 instances.

Queue topology and connection locality

The ingest layer accepts swipes independently of where WebSocket sessions live. A separate RabbitMQ EC2 routes raw events through **skitrak.exchange** to ten durable resort queues. A separate consumer EC2 computes standings and publishes full snapshots to **leaderboard.topic**, using exact keys **lb.1 ... lb.10**. Each WebSocket server owns an exclusive generated queue and binds only resorts with local subscribers.

The topic backplane allows ingest/WebSocket nodes to scale independently of aggregation. Co-location would save a hop but couple resort routing, state ownership and node capacity. Selective topic bindings save uninterested traffic, at the cost of managing subscription transitions and bootstrap snapshots.

Subscription lifecycle and ordering

First subscription: install the resort binding, then fetch the authoritative snapshot. Bootstrap responses and live updates reconcile by persistent **(resort, day, version)**. Last unsubscribe removes the binding; stale bootstrap responses are ignored. Broker reconnect recreates the private queue, restores active bindings and refreshes snapshots. Periodic refresh repairs transient notification loss. Each session has one asynchronous send in flight and a bounded queue; slow sessions reconnect for a fresh snapshot.

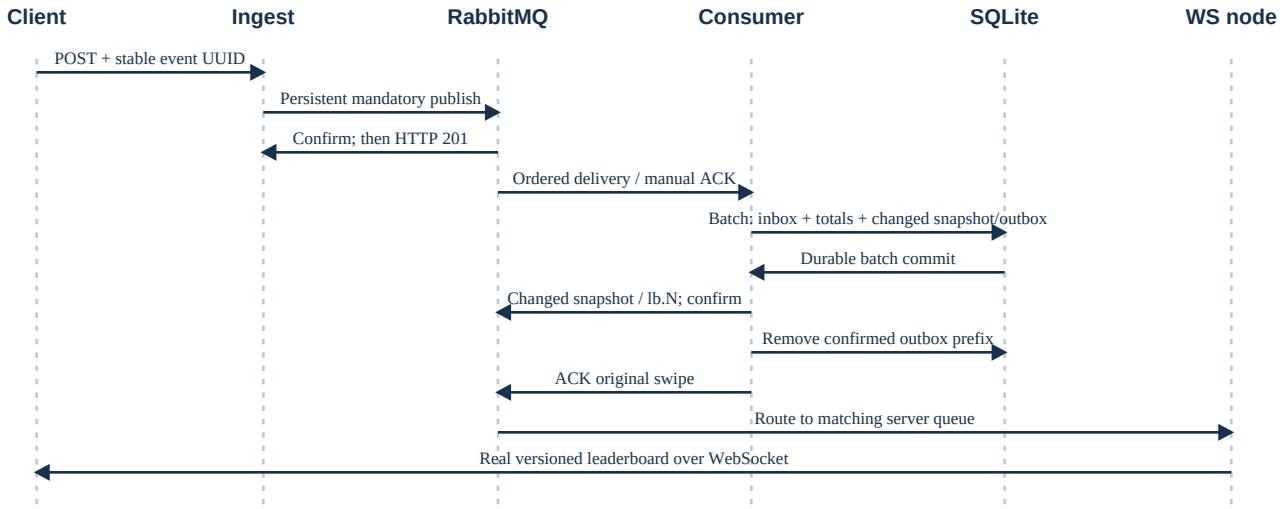
Consumer threading and Top-N

One FIFO serial lane owns each resort. A shared executor yields between batches for fair scheduling. Automatic scaling is supported; fixed worker counts isolate the 5/10/15/20 sweep. Ten queues cap useful aggregation parallelism at ten. The instructor-endorsed sweep is **5/10/15/20**, with expected flattening beyond ten useful lanes. Retry stays within the same lane; invalid/conflicting events pause that partition rather than let later events overtake it.

All-skier totals plus a bounded ordered set maintain Top-20 incrementally. Scores sort descending with numeric skier-ID ties. A changed displayed score also triggers a broadcast, even when rank order is unchanged.

Delivery, recovery and verification

Swipe-to-leaderboard sequence and operational guarantees



Reliable publication and durable aggregation

A client generates one UUID per logical swipe and preserves it through retries. Ingest validates the request, looks up the supplied official lift vertical, borrows a pre-created Channel exclusively, and returns 201 only after mandatory routing and publisher confirmation. One producer connection backs the bounded pool; one recovery owner replaces a failed connection and its channels. Old leases cannot re-enter a replacement pool. Unknown outcomes return 503 and retain the same event ID on retry.

Each resort uses SQLite WAL with FULL synchronization. Up to 64 events commit atomically per batch, with a 10ms partial-batch flush trigger. Inbox and totals always persist; snapshot/version and outbox change only when Top-N changes. All changed snapshots retain order; grouped publisher confirms precede source ACK (batch 16, prefetch 128). A confirmed outbox prefix is then deleted; a crash can repeat a snapshot without recounting the swipe. No matching notification queue is normal when nobody subscribes. Raw queues default to 24h TTL, capacity 100000 and reject-publish overflow; the operating depth target remains below 1000. Expired/dead-lettered events are not successful aggregation.

ALB and process recovery

ALB: round robin, generated-cookie stickiness=300s, **/health** at 30s intervals, 5s timeout, healthy=2, unhealthy=3, idle timeout=120s. ROOT WAR makes the health path literal. WebSocket Upgrade pins a live socket to its accepted node; node failure requires client reconnect and snapshot bootstrap. Docker/systemd restart processes. Broker and consumer access use private-network rules; management stays behind loopback/SSH tunneling.

Evidence and design boundaries

Completed: 61 local regression tests; real RabbitMQ duplicate/restart tests; EC2 500K runs at 1/2/4 ingests; four-node 1M stress; client 64/128/256/512 and consumer 5/10/15/20 sweeps. Functional tests verify official-table totals and real WebSocket snapshots. ALB probes show even distribution and cookie stickiness. A Python harness reconnects from failed ingest-3 to ingest-4 in 2.981s, then restores all four targets.

Four ingests / 128 client threads / 20 fixed workers: 500K completed, zero retries, sampled raw-queue total peak 253, mean consumer lag 25.61ms. At 512 threads / 1M: ready peak 417048, mean lag 52.68s; high-load targets were missed. Earlier EBS IOPS checks show storage pressure, not an exclusive cause. Lag means and sampled peaks are not per-event guarantees. State survives process restart on retained EBS; host/disk-loss HA is outside this single-broker/consumer design.